

7.2. Проектирование, плавно переходящее в конструирование

В качестве примера попробуем описать процесс детального проектирования системы учета рейтинга успеваемости. В 4.1 ее архитектура была показана без погружения в подробности, на уровне слоев, компонент и их связей. Дальнейшее продвижение содержит определенное противоречие. С одной стороны, уровень проработки не позволяет писать код, с другой – проработка, оторванная от реализации, должна вестись на уровне документов, на которые у программиста душа не лежит.

Выходом является проектирование *от кода* (см.1.1), которому здесь самое место. Детальное проектирование сопровождается тремя видами артефактов:

- каркас программного проекта: интерфейсы, абстракции, заготовки классов - свойства, структуры данных, заголовки ключевых методов, наследование, функциональные спецификации классов в комментариях;
- минимальный набор формальных документов - диаграммы архитектурных классов, диаграммы для ключевых взаимодействий архитектурных компонент;
- словесные формулировки ключевых моментов, структур и взаимодействий, применяемых паттернов.

Результатом дальнейшего наполнение каркаса кодом является появление архитектурного прототипа, либо прототипов отдельных подсистем или компонент.

Уровень доступа к данным. Классы DAO

SQL-запросы не используются на уровне бизнес-логики. Единственным пользователем коннектора к БД являются объекты уровня доступа к данным (Data Access Object - DAO). Сервис доступа данным реализован в классе DBItem с помощью рефлексии:

- любому классу, производному от DBItem, ставится в соответствие таблица в присоединенной БД с именем, совпадающим с именем класса без префикса DB;
- методы выбора, добавления или обновления DAO-объекта в таблице БД по его ключу-идентификатору, а также создание таблицы обеспечивают однозначное соответствие имен и типов данных класса и его предков с именами полей таблицы;
- поддерживаются соглашения для имен ссылочных полей со связью типа «один ко многим» и получение вектора DAO-объектов по значению связываемого поля;
- модификатор transient запрещает связывать свойство DAO-объектов с полем БД.

Классы табличных объектов соответствуют основным классам-сущностям модели предметной области, а ссылочные поля – отношениям между классами (рис.7-5).

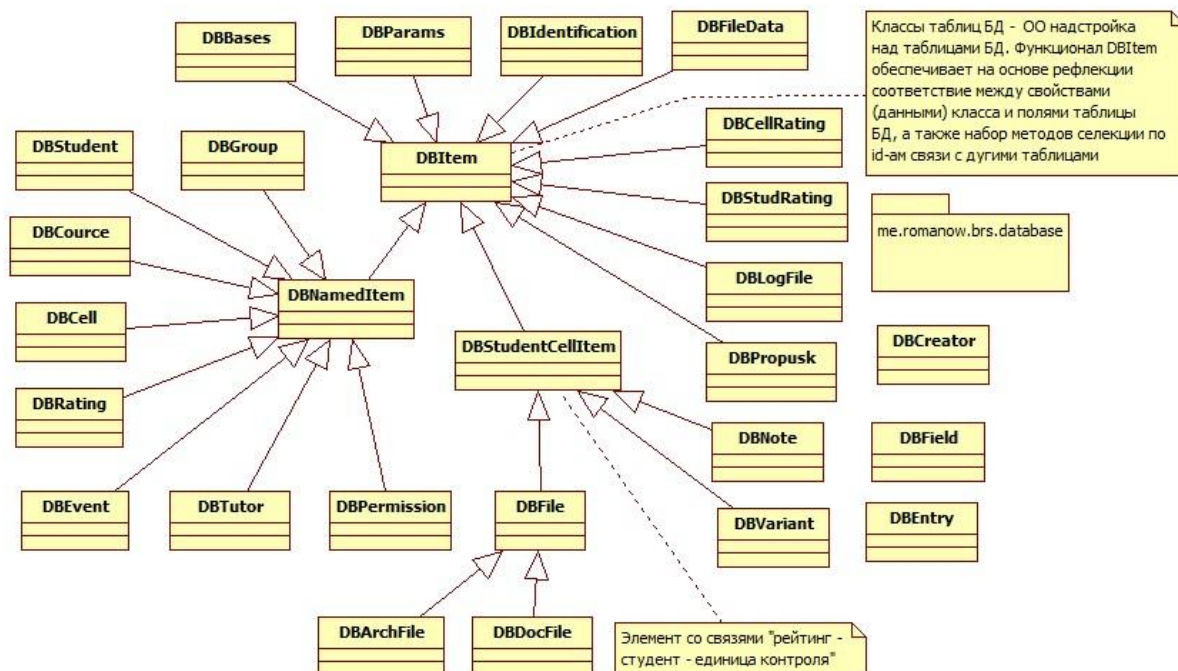


Рис. 7-5. Диаграмма классов таблиц БД (DAO)

Структура БД, изображенная на 7-6, генерируется автоматически по списку классов DAO-объектов приложения.

Group	Rating	Event *	Entry (Profile)**	Variant (x)
id	id	id	id	id
name	name	name	name	idStudent
	idGroup	idRating	user	idRating
	idCourse	evtDate	pass	idCell
	second	removed	ip	cDate
	idParams		port	variant
			selected	
Student		Propusk *** (x)		DocFile/ArchFile (x)
id		id		id
name		idStudent		idStudent
idGroup		idRating		idRating
ciuLogin		idEvent		idCell
pass		removed		cDate
		cDate		fileId
				fileName
Course	Params		Tutor	
id	id		id	
name	maxWeek		name	
	weekProc		pass	
	propuskBall		ciuName	
	plusProc			
	semestr		Permission	
	tutorId		id	
Cell	StudRating (*)	CellRating (*)	name	Note *** (x)
id	id	id	idTutor	id
name	idStudent	idRating	idRating	idStudent
idCourse	idRating	idCell		idRating
cType	brigade	week		idCell
ball	second	week2	FileData	ball
ordNum	cDate	cDate	id	week
			data	params
			idFileData	removed
				cDate

* - заполняются при добавлении рейтинга по его элементам
 ** - файл brs_config.dat
 *** - множественные экземпляры записей для архива операций
 x - избыточное поле idRating для восстановления контекста

Рис. 7-6. Структура БД

Для создания таблицы в БД достаточно определить класс-наследник для DBItem со списком необходимых свойств – примитивных типов данных и свойств – ссылочных полей.

Классы бизнес-объектов

Классы бизнес-объектов (см.4.1) состояются из связанных объектов уровня DAO таким образом, чтобы следующий уровень бизнес-процессов получал уже готовые сборки бизнес-сущностей с необходимыми, часто используемыми связями. Принципы создания бизнес-объектов данной реализации:

- классы бизнес-сущностей наследуют соответствующие DAO-объекты, например, *MDStudent* - *DBStudent*. Бизнес-сущности могут содержать поля, загружаемые из других источников, например, из связанных таблиц.

```
public class MDStudent extends DBStudent{  
    transient public boolean studRatingChanged=false;  
    public int brigade=0; // Данные из связанной таблицы StudRating  
    public int cDate=0;  
    public boolean second=false;
```

Рис. 7-7. Бизнес-объект, унаследованный от DAO

- бизнес-объекты могут содержать векторы связанных бизнес-сущностей. Для их загрузки соответствующему методу необходимо передать класс связываемых DAO-объектов и DAO-объект, на который они ссылаются. Последний содержащий требуемое значение ключа. Например, для получения вектора объектов – студентов группы необходимо передать объект *DBGroup* со значением ключа и описатель класса *DBStudent.class*. Генерируется SQL-запрос в соответствии с принятыми соглашениями по именованиям связываемых полей;
- система бизнес-объектов построена по иерархическому принципу (рис.7-8). Основной бизнес-объект – рейтинг (*MDRating*) содержит бизнес-объекты группа (*MDGroup*) и предмет (*MDCourse*), которые имеют векторы связанных бизнес-объектов – студент (*MDStudent*) и единица контроля (*MDCell*) соответственно.

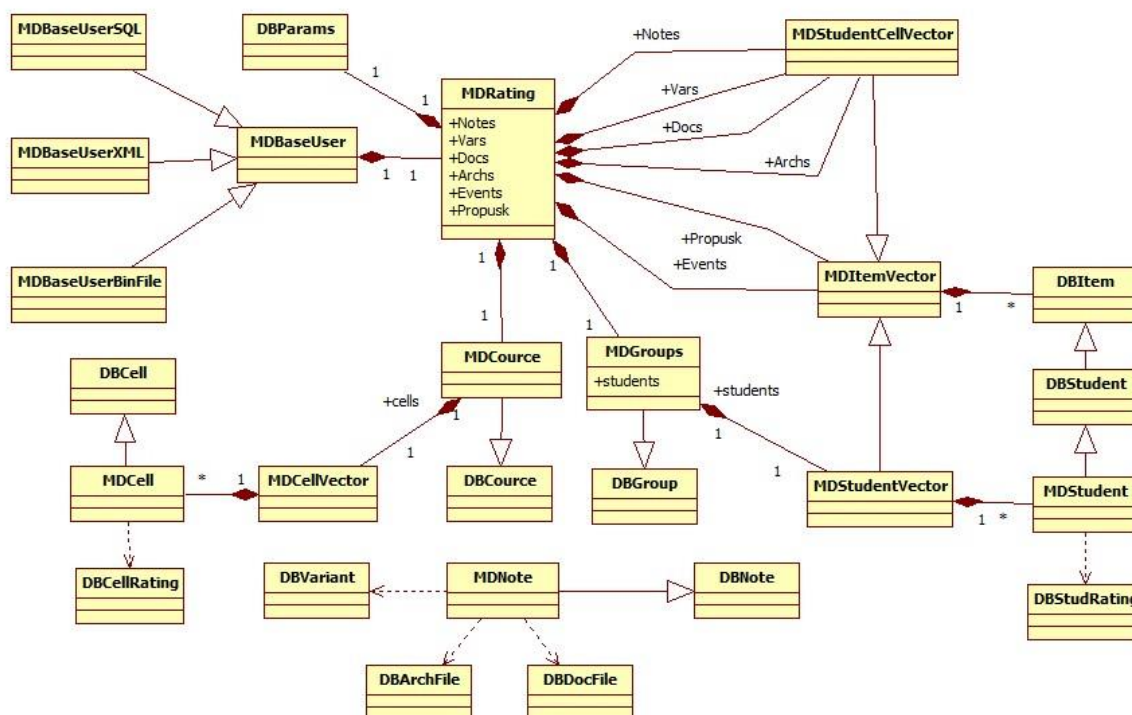


Рис. 7-8. Диаграмма классов бизнес-модели

```
public class MDRating extends DBRating{
    public MDGroups groups=null;
    public MDCourse course=null;
    public DBParams params=null;
    public MDItemVector events=new MDItemVector();
    public MDItemVector propusk=new MDItemVector();
    public MDStudentCellVector notes=new MDStudentCellVector();
    public MDStudentCellVector vars=new MDStudentCellVector();
    public MDStudentCellVector docs=new MDStudentCellVector();
    public MDStudentCellVector archs=new MDStudentCellVector();
}
```

Рис. 7-9. Заголовок класса MDRating

- бизнес-объект *рейтинг* работает с множествами данных, существующих в *двумерных системах координат - студент/единица контроля, студент/занятие*. Иначе говоря, табличных данных с соответствующими координатами осями. К таковым относятся: оценки, пропуски, файлы отчетов и архивов. Для моделирования такого представления имеются специальные классы, использующие линейные векторы DAO-объектов с соответствующими парами ключей;
- для бизнес-объекта *рейтинг* имеется три метода загрузки данных: *минимальный, стандартный и полный* с соответствующей глубиной связывания данных по иерархии.
- бизнес-модель реализована в классе *MDBaseUser* с интерфейсом *MDBaseFace*, который является базовым для расширений – источников данных модели (рис.7-10,7-11).

Рис. 7-10. Интерфейс MDUserFace

```
public abstract class MDBaseUser extends DBNamedItem implements MDBaseFace{
    //----- Компоненты данных -----
    public MDRating rating=null;           // бизнес-объект - рейтинг
    public DBTutor tutor=null;             // бизнес-объект - преподаватель
    public DBProfile entry=null;           // бизнес-объект - данные подключения
    //----- Компоненты редактирования -----
    public DBTutor []permission=null;      // массив id-ов разрешений для преподавателя
    public DBItem []ratingList=null;       // массив id-ов списка рейтингов
    public boolean editEnabled=false;      // разрешение редактирования рейтинга
    public MDStudent studentItem=null;     // бизнес-объект - выбранный студент
    public MDCell cellItem=null;           // бизнес-объект - выбранная единица контроля
    public DBEvent eventItem=null;         // бизнес-объект - контроля посещаемости (занятие)
    public boolean wasChanged=false;       // флаг изменения рейтинга
    public DBConnect getConnect(){ return null; }
}
```

Рис. 7-11. Класс MDBaseUser

Источники данных для бизнес-модели

Производные классы бизнес-модели (рис.7-12) обеспечивают реализацию толстого и тонкого клиентов (WebAPI). Базовый класс бизнес-логики *MDBaseUser* имеет три расширения:

- *MDBaseUserSQL* загружает бизнес-объекты непосредственно через соединение с БД, используется в толстом клиенте;

- *MDBaseUserBinFile* сохраняет бизнес-объект *рейтинг* в двоичном файле, используется для работы с локальными копиями рейтингов;
- *MDBaseUserXML* реализует режим *тонкого клиента* с созданием WebAPI. Используется ряд классов, сериализуемых в XML-формат, для поддержки команд и ответов протокола (*XMLCmd*, *XMLAnswer*, *XMLArray*). В протоколе также сериализуются бизнес-объекты, например *MDNote*. *MDBaseUserXML* является клиентской частью протокола, а *MDXMLCommand* - серверной. Последний создает на сервере объект *MDBaseUserSQL* и выполняет в нем необходимые команды от имени клиента.

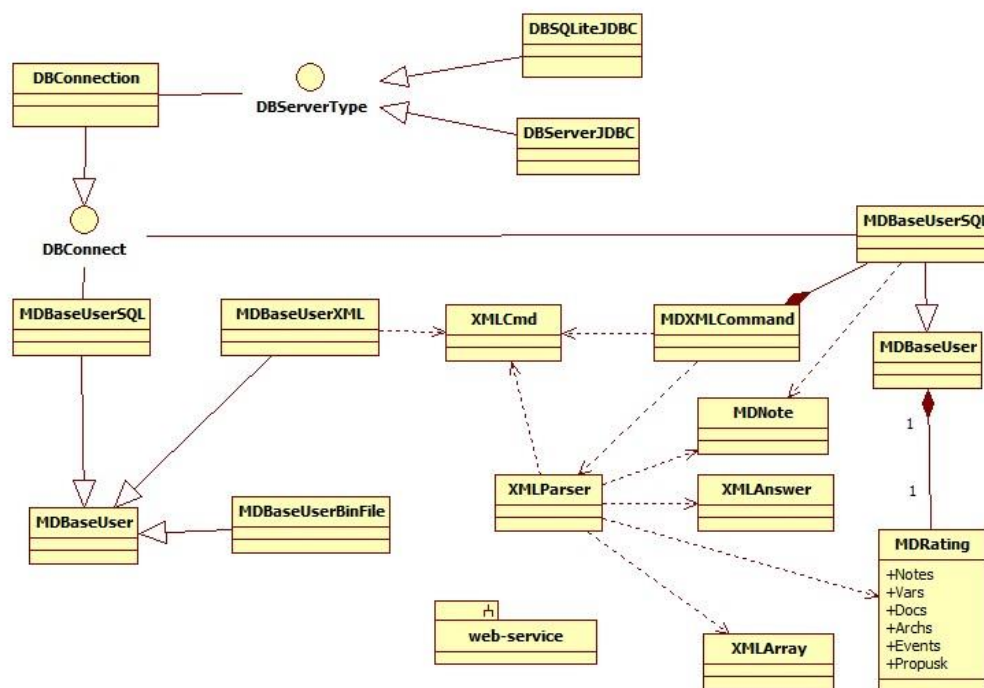


Рис. 7-12. Производные классы для источников данных

Реализация толстого клиента

В толстом клиенте сетевые коммуникации реализованы непосредственно в драйвере доступа к БД JDBC, поэтому все слои приложения локализованы в клиенте. Производный класс бизнес-модели *MDBaseUserSQL* (рис.7-13) доопределяет абстрактные методы загрузки бизнес-объектов через прямое обращение к объекту соединения с БД *DBConnect*, поддерживающему сервис DAO.


```

public class MDBaseUserSQL extends MDBaseUser{
    public final static int blockSize=32000;
    private DBConnect conn=null;
    public DBConnect getConnect(){ return conn; }

    @Override
    public void loadFullRating(int id) throws Throwable{
        loadShortRating(id);
        loadStudentList();
        loadCellList();
        loadEventList();
        loadPermissionList();
        //-----
        DBItem xx[]=conn.getList(DBNote.class, rating);
        rating.notes.clear();
        for(int i=0;i<xx.length;i++){
            rating.notes.add(xx[i]);
        }
        xx=conn.getList(DBVariant.class, rating);
        rating.vars.clear();
        for(int i=0;i<xx.length;i++){
            rating.vars.add(xx[i]);
        }
    }
}

```

Рис. 7-13. Производный класс MDBaseUserSQL толстого клиента

Протокол и коммуникации в тонком клиенте с WebAPI

На рис.7-12 изображены также классы коммуникаций для тонкого клиента. Прикладной протокол тонкого клиента строится на базе http-протокола и содержит набор синхронных запросов к серверу, соответствующих абстрактным методам загрузки и обновления данных бизнес-модели в *MDBaseUser*.

Далее процесс фактически переходит от детального проектирования к конструированию, дальнейшая разработка протокола элементарно вписывается в технологические средства поддержки протокола на Java классами *URLConnection* и *Servlet*:

- клиент создает http-соединение на основе объекта класса *URLConnection*;
- часть параметров команды клиента может быть передана в заголовке http-запроса;
- после передачи заголовка клиент и сервер договариваются, в каком формате - текстовом или двоичном, а также в какой кодировке они будут передавать дополнительные данные запроса и ответ. Аналогичные параметры имеются в самом заголовке http-запроса, поэтому их необходимо корректно устанавливать;
- обычно в качестве передаваемых данных используются объекты, сериализованные в двоичном или общепринятом текстовом формате XML или JSON;
- описание протокола состоит из описания форматов запросов клиента и возможных ответов сервера;

Решения, принятые далее, являются технологическими: форматы представления потоков данных, проблемы размерностей стандартных типов данных (String в формате UTF), кодирование команд и ответов, способ сериализации объектов, *прикрепление* объектов данных в ответе сервера.

Единственным принципиальным решением является создание на сервере класса бизнес-модели *MSBaseSQL* с локальным источником данных, что позволяет использовать общий код слоя доступа к данным в тонком и толстом клиенте.

Клиентская часть протокола

От клиента к серверу не передается значительных объемов данных, поэтому большинство запросов оформляется в виде объекта класса *XMLCmd* (рис.7-14), при необходимости вслед за ним в потоке передается дополнительный объект.

```
public class XMLCmd {  
    public int code=0;  
    public int id=0,id2=0,id3=0,id4=0;  
    public String str="";  
}
```

Рис. 7-14. Класс – запрос клиента

В нашем случае используется XML-сериализация объектов, передаваемых в запросе или ответе. В первом варианте используются **двоичные потоки данных** (рис.7-15), для передачи запроса объект *XMLCmd* и дополнительный объект данных сериализуются, а полученные строки пишутся методом *writeUTF* (строка в смешанной кодировке UTF, предваренная счетчиком байтов).

Ответ сервера выглядит аналогично. Вначале следует объект *XMLAnswer* с основными параметрами ответа. В протоколе предусмотрены три варианта в соответствии с кодом ответа:

- ответ без дополнительного объекта данных;
- ответ с дополнительным объектом данных, возвращаемый в качестве результата метода;
- сообщение об ошибке, приводящее к генерации исключения в клиенте.

```
private Object sendCmd(int code, int id, int id2, int id3, int id4, String w, DBItem obj) thro  
Object res=null;  
try {  
    if (binary) { // Двоичный поток туда-обратно  
        httpConnect();  
        XMLCmd cmd=new XMLCmd (code,id,id2,id3,id4,w);  
        DataOutputStream out=new DataOutputStream (conn.getOutputStream());  
        out.writeUTF (pars.toXML (cmd)); // Сериализованный XMLCmd в двоичном формате  
        if (obj!=null) {  
            out.writeUTF (pars.toXML (obj)); // Дополнительный объект в двоичном формате  
        }  
        out.flush(); // Протолкнуть данные в поток  
        is=conn.getInputStream(); // Открыт двоичный поток приема  
        DataInputStream dis=new DataInputStream (is);  
        ans=(XMLAnswer) pars.fromXML (dis.readUTF()); //  
        if (ans.code>0) throw new BRSEException (BRSEException.serv,ans.message);  
        if (ans.code==XMLAnswer.noobj) res=null;  
        else { // Есть присоединенный объект  
            int sz=dis.readInt(); // Счетчик длины (int)  
            char cc[]=new char[sz]; // Массив принимаемых символов  
            for (int i=0;i<sz;i++) cc[i]=dis.readChar();  
            String ss=new String(cc); // Строка на основе принятого массива  
            res=pars.fromXML (ss); // Парсить XML-ответ из строки  
            if (res instanceof XMLAnswer)  
                throw new BRSEException (BRSEException.serv, ((XMLAnswer) res).message);  
        }  
    }  
}
```

Рис. 7-15. Клиент с двоичным потоком данных

Во втором варианте используется текстовый поток в байтной кодировке *Cp1251*. При передаче все параметры вносятся в заголовок http-запроса в виде параметров URL-строки в формате *&value*.

Ответ сервера передается в текстовом формате с байтной кодировкой, вначале следует строка с сериализованным объектом *XMLAnswer*, в качестве ограничителя строки используется стандартный символ «\n». Чтение производится из потока *InputStreamReader*, который делает перекодировку принимаемых символов во внутренний формат *Unicode*. Если далее следует дополнительный объект, что определяется из параметров *XMLAnswer*, то он создается и заполняется библиотекой непосредственно из открытого потока (рис.7-16).

```
String req="code="+code;           // Сформировать строку http-запроса с параметрами
if (id!=0) req+="&id="+id;
if (id2!=0) req+="&id2="+id2;
if (id3!=0) req+="&id3="+id3;
if (id4!=0) req+="&id4="+id4;
if (w!=null && w.length() !=0) {
    w=URLEncoder.encode(w, "Cp1251");
    req+="&str="+w;
}                                // Добавить параметры из дополнительного объекта
if (obj!=null) req+=obj.saveURLParams();
httpConnect(req);
wr=new OutputStreamWriter(conn.getOutputStream(), "Cp1251");
wr.flush();
is=conn.getInputStream();         // Открыть текстовый поток ответа
rd=new InputStreamReader(is, "Cp1251");
int nn=0;
String hh="";
//----- Прочитать СТРОКУ ответа, т.к. парсер из потока БЕРЕТ ВСЕ - свое и чужое -----
while( (nn=rd.read()) !='\n' && nn!=-1)
    hh+=(char)nn;                // Читать символы по получения «конца строки»
if (nn==-1) throw new BRSEException(BRSEException.bug, "Обрыв команды:"+hh);
ans=(XMLAnswer) pars.fromXML(hh); // Парсить ответ в объект
if (ans.code>0) throw new BRSEException(BRSEException.serv, ans.message);
if (ans.code==XMLAnswer.noobj) res=null;
else{                            // В потоке есть дополнительный объект
    res=pars.fromXML(rd);         // Парсить непосредственно из потока
    if (res instanceof XMLAnswer)
        throw new BRSEException(BRSEException.serv, (XMLAnswer)res).message;
}
} catch(Throwable ee) {
    conn.disconnect();
    throw new BRSEException(ee);
}
conn.disconnect();
return res;
```

Рис. 7-16. Клиент с текстовым потоком данных

В большинстве случаев реализация переопределяемых методов получения и изменения бизнес-объектов на клиентской стороне заключается в вызове единственного метода *sendCmd* с передачей ему параметров и сохранением объекта-результата (рис.7-17).


```

@Override
public void loadRating(int id) throws Throwable {
    rating= (MDRating) sendCmd (XMLCmd.loadRating, id) ;
}

@Override
public void loadFullRating(int id) throws Throwable {
    rating= (MDRating) sendCmd (XMLCmd.loadFullRating, id) ;
}

```

Рис. 7-17. Получение бизнес-объектов на стороне клиента

Серверная часть протокола

Из описания протокола в клиенте понятно, как аналогичные действия будут выглядеть на стороне сервера (рис.7-18). Сервлет читает объект *XMLCmd*, создает парсер и локальную бизнес-модель, после чего переключается на обработку команды с соответствующим кодом. Перехватываемые исключения отправляются в виде объекта *XMLAnswer* с кодом и текстом ошибки.

```

protected void processRequestBinary(HttpServletRequest request, HttpServletResponse response) {
    out=null;
    XMLParser pars=null;
    XMLCmd cmd=null;
    try {
        pars=getParser();
        response.setContentType("text/html;charset=UTF-8");
        out=new DataOutputStream(response.getOutputStream());
        DataInputStream is=new DataInputStream(request.getInputStream());
        cmd=(XMLCmd) new XMLParser().fromXML(is.readUTF());
        MDBaseUserSQL conn=getJDBC();
        switch (cmd.code) {
default:      throw new BRSEException(BRSEException.serv,
        "функция код:"+cmd.code+" не поддерживается");
        .....
    } catch(Throwable ee) {
        try {           // исключения – объект XMLAnswer с кодом ошибки и сообщением
            BRSEException e2=new BRSEException(ee);
            XMLAnswer ans=new XMLAnswer(e2.getCode(), e2.getMessage());
            out.writeUTF(pars.toXML(ans)); // передать объект в качестве ответа
        } catch(Throwable e3) {}
    }}
}

```

Рис. 7-18. Серверная часть протокола для двоичного потока

Типовая реализация серверной компоненты предусматривает наличие *агента*, который исполняет от имени удаленного клиента локальные запросы в соответствии с принятой командой и отправляет полученные ответы. В его функции также входит поддержание необходимого *контекста*, в котором исполняются локальные запросы.

В нашем случае никакого постоянного окружения запросы не имеют, т.к. являются независимыми друг от друга. Поэтому сервер просто вызывает соответствующий метод в полученной локальной бизнес-модели (*MDBaseUserSQL*), а затем сериализует объект ответа *XMLAnswer*, а вслед за ним необходимую компоненту, извлеченную из бизнес-модели. Если в команде присутствует дополнительный объект, то он читается непосредственно обработчиком команды (рис.7-19).

```

private void writeChars (String ss) throws Throwable{ // Передача строки неограниченной длины
    int sz=ss.length () ;                               // в двоичном формате
    out.writeInt (sz) ;                                  // Передать счетчик длины
    char cc []=ss.toCharArray () ;                     // Конвертировать строку в массив символов
    for (int i=0 ; i<sz ; i++)
        out.writeChar (cc [i] ) ;                      // Передать символы
    }

.....
case XMLCmd.loadRating :                               // Читать рейтинг по id
    conn.loadRating (cmd.id) ;                          // Загрузить в локальную бизнес-модель
    out.writeUTF (pars.toXML (new XMLAnswer (true)) ) ;
    writeChars (pars.toXML (conn.rating) ) ; // Передать заголовок ответа XMLAnswer
    break ;                                              // с отметкой, что следует объект данных
case XMLCmd.loadFullRating :                           // Сериализовать бизнес-объект MDRating
    conn.loadFullRating (cmd.id) ;
    out.writeUTF (pars.toXML (new XMLAnswer (true)) ) ;
    writeChars (pars.toXML (conn.rating) ) ;
    break ;
case XMLCmd.changeNote :                               // Редактировать оценку
    conn.loadShortRating (cmd.id) ;                    // Загрузить «компактную» бизнес-модель
    conn.setStudentItem (cmd.id2) ;                    // Установить студента и ед.контроля
    conn.setCellItem (cmd.id3) ;                       // по переданным id-ам
    // Читать дополнительный объект напрямую из потока
    MDNote mm=(MDNote) new XMLParser () .fromXML (is.readUTF () ) ;
    conn.changeNote (mm) ;                             // Вызвать метод в модели с параметром
    out.writeUTF (pars.toXML (new XMLAnswer () ) ) ;   // - полученным объектом
    break ;

```

Рис. 7-19. Непосредственная обработка запросов сервером

С приведенным кодом связан один из *типовых дефектов диапазона представления данных* (см.8.2), который вручную нейтрализован с помощью метода `writeChars`.

В качестве иллюстрации приведем диаграммы последовательности для толстого и тонкого клиентов. Даже поверхностное визуальное сравнение позволяет увидеть идентичность поведения на стороне клиента и на стороне сервера в первом и втором случае (рис.7-20,7-21).

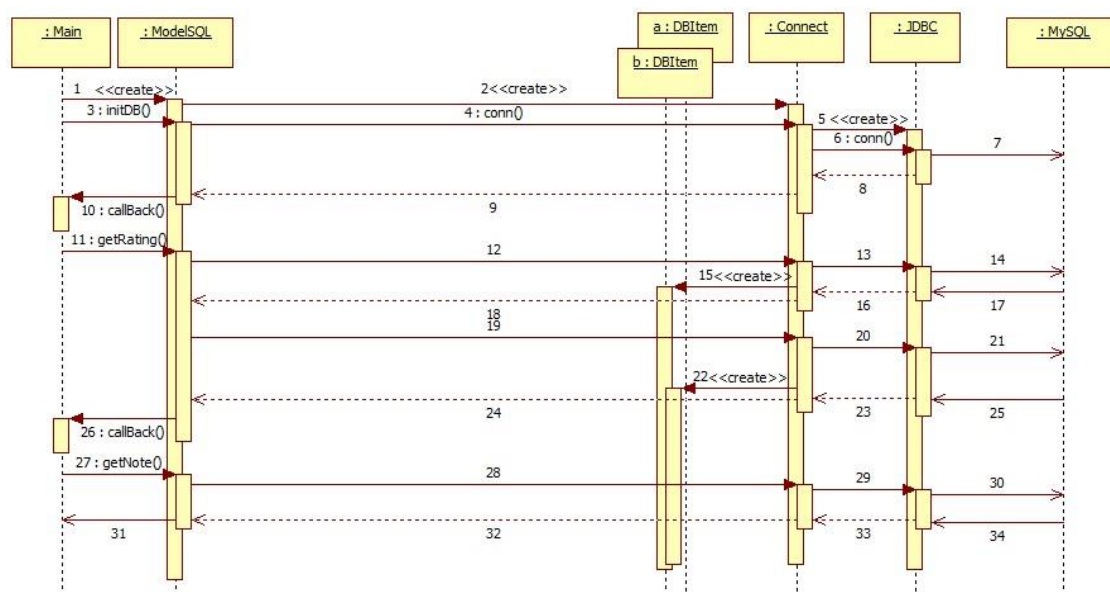


Рис. 7-20. Толстый клиент. Сетевое соединение в библиотеке доступа к БД

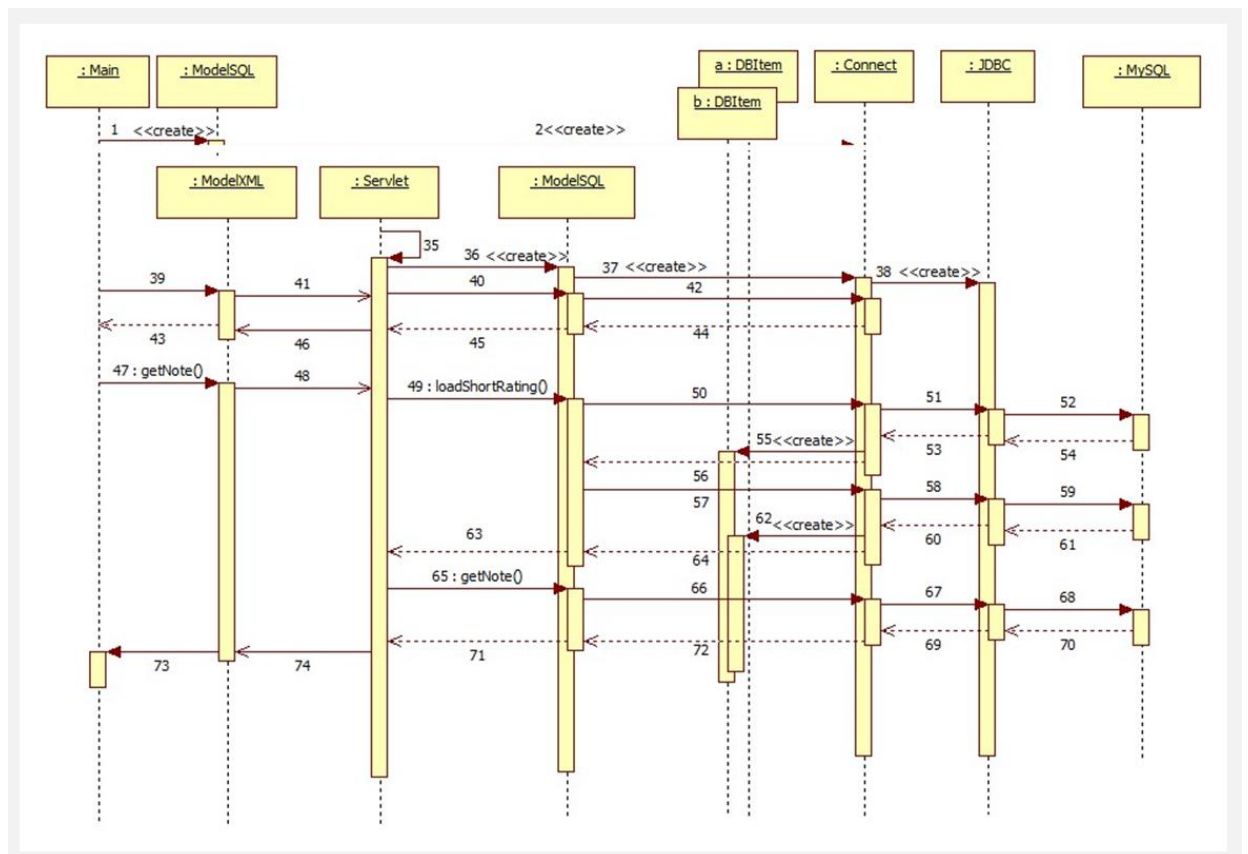


Рис. 7-21. Тонкий клиент. Сетевой протокол в бизнес-логике

Архитектурный MVC

В 4.2 были описаны особенности паттерна архитектурного MVC, в котором контроллер определяет долгосрочное поведение представления в бизнес-процессе. Одним из преимуществ такого подхода является возможность повторного использования кода контроллера в последовательности форм на разных платформах. В нашем примере бизнес-процесс определяется возможностью управления рейтингом и просмотра его составляющих путем выбора соответствующих полей из списков и нажатия кнопок.

Контроллер реализует варианты поведения внешнего представления, определяет возможные последовательности *нажатия кнопок* и генерирует события для изменения состояния *элементов отображения*. Он реализует модель поведения на основе диаграммы состояний (рис.7-22).



Рис. 7-22. Диаграмма состояний внешнего представления в контроллере

Контроллер является имитатором поведения абстрактного представления. В идеальном случае представление – тонкий клиент, лишенный модели поведения. Контроллер и представление связаны двумя интерфейсами – передачи *событий* от представления к контроллеру и *команд* контроллера по вводу/выводу содержимого и отображению элементов управления. Бизнес-модель (*MDBaseUser* и его производные) –

создается и управляется контроллером, хотя некоторые ее данные могут быть получены представлением по прямой ссылке (рис.7-23).

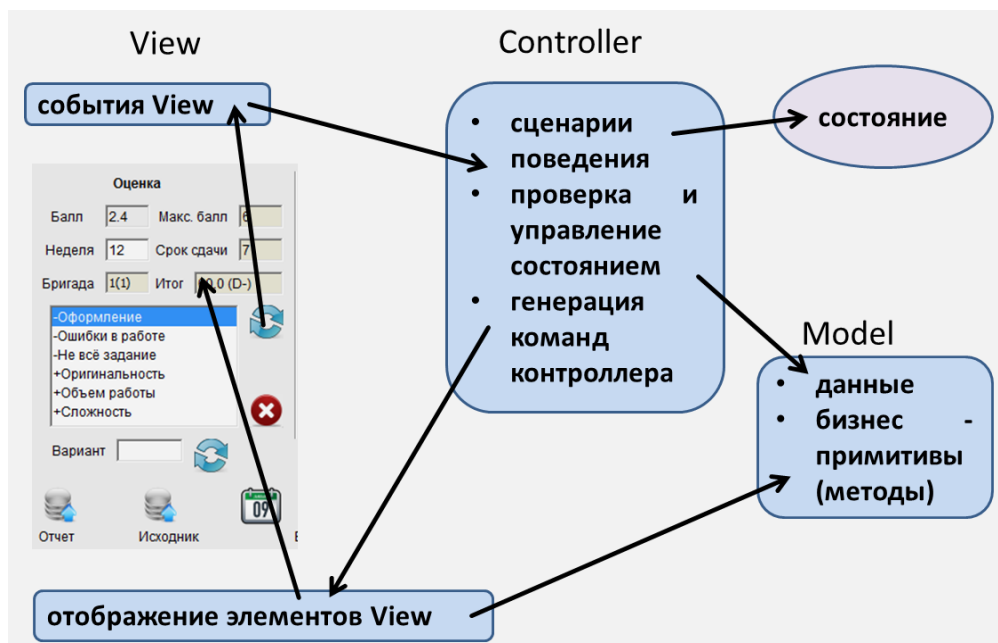


Рис. 7-23. Взаимодействие представления (View) и контроллера

Интерфейс *ViewControllerCommands* определяет набор событий и команд, передаваемых контроллеру со стороны представления, инициируемых такими действиями как нажатие кнопок или выбор элементов списков (рис.7-23).

```

public interface ViewControllerCommands {
    public double getSum();
    public MDNote getNoteltem(); // Получить текущую оценку от модели
    public void setState();
    public void retryState();
    public void close();
    public void updateAllLocalRatings(); // Синхронизировать локальные копии
    public void loadRating(int id) throws Throwable; // Загрузить рейтинг из модели
    public void loadFullRating(int id) throws Throwable;
    public void loadFullRatingSoft(int id) throws Throwable;
    public void setState(int newState); // Попытка перевести контроллер (модель) в состояние
    public void callBacs(); // «Провоцирование» контроллера на возможные события
}

```

Рис. 7-24. Интерфейс событий и команд представления

Интерфейс *ViewControllerListener* определяет набор команд контроллера для выполнения представлением действий над элементами отображения (рис.7-25).

```

public interface ViewControllerListener {
    public String getInputFileName (String title, String defName) throws Throwable;
    // получение имени файла контроллером
    public String getOutputFileName (String title, String defName) throws Throwable;
    public void fatal (Throwable ee); // исключение, перехваченное контроллером
    public void stateChanged (int state); // изменение состояния контроллера
    public void setRatingVisible (boolean enb); // «видимость» полей данных рейтинга
    public void setCellVisible (boolean enb); // «видимость» полей данных единицы контроля
    public void setStudentVisible (boolean enb); // «видимость» полей данных студента
    public void setNoteVisible (boolean enb); // «видимость» полей данных оценки
}

```

Рис. 7-25. Интерфейс команд контроллера

Контроллер управляет всем бизнес-процессом приложения, поэтому он создается главным окном и передается по ссылке между окнами (формами).

Для управления поведением контроллер имеет методы «провоцирования» (повторной генерации) текущих событий, проверки текущего состояния бизнес-процесса, попытки его перевода в указанное состояние, а также пропускает через себя команды от представления по изменению компонент модели, проверяя возможность исполнения соответствующих действий.

Помимо основных функций, контроллер в таком виде может выполнять другие общесистемные функции:

- кэширование данных - создание дополнительной бизнес-модели, настроенной на локальный источник данных. Эта бизнес-модель используется при отсутствии соединения с сервером, невидима для представления. При восстановлении соединения с сервером контроллер синхронизирует накопленные в ней изменения с основной моделью;
- перехват исключений, возникающих в модели, представлении, а также генерируемых самим контроллером, информирование о них представления через интерфейс событий контроллера в текстовом формате.

Рис.7-26 демонстрирует, что программный код представлений является *чистым* с точки зрения поведения, т.е. представляет собой формальную переадресацию событий и связанных с ними данных от элементов управления формы к контроллеру и обратно с использованием платформенно-зависимых методов работы с графическим интерфейсом в данном приложении.



Рис.7-26. Реализация интерфейсов контроллера в desktop и Android

Тонкий web-клиент

Рассмотрим еще один вариант тонкого клиента – клиент как набор html-страниц web-сервера (web-клиент). Для эффективной реализации необходимо подобрать технологию, максимально сохраняющую преемственность с текущей архитектурой и структурой кода. В качестве такой технологии можно использовать Ajax [47-6].

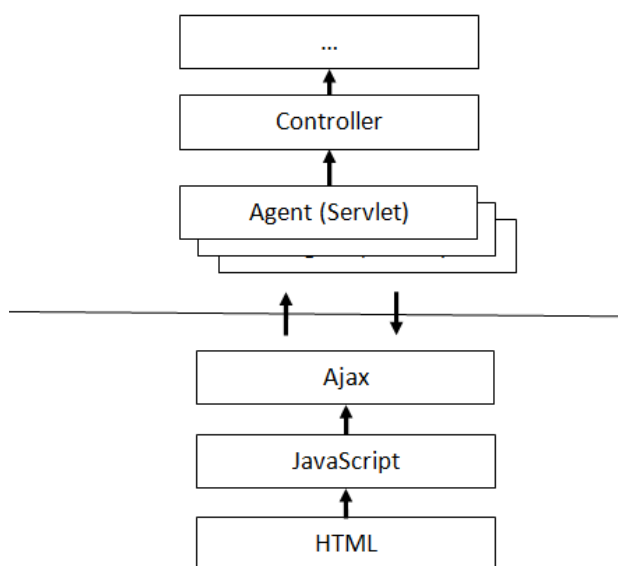


Рис. 7-27. Технология Ajax в web-клиенте

Ајах позволяет посылать отдельные запросы к серверу, не привязанные процессу перерисовки web-страницы. Клиентская компонента протокола посылает собственный запрос к серверу, указывая необходимый активный ресурс, например, сервлет. Активный ресурс возвращает ответ в формате XML, из которого клиентская компонента извлекает необходимые данные. Применительно к описываемой архитектуре этот процесс будет выглядеть так:

- существующая разметка формы тонкого клиента преобразуется в разметку web-страницы (HTML);
- на активные элементы страницы (кнопки, поля со списком), навешиваются события, обработчики которых реализуются на JavaScript;
- каждый обработчик посылает Ајах-запрос к сервлету с кодом команды и параметрами;
- сервлет по коду команды выполняет соответствующее обращение к контроллеру на стороне сервера, полученные ответные данные передаются клиенту сообщением в формате XML;
- поскольку контроллер запоминает текущее состояние представления, то на стороне сервера необходимо использовать механизм сессий для сохранения контекста соединения [47-7].

В оригинале интерфейс представление-контроллер является двунаправленным, т.к. ответы контроллера формально выглядят как *асинхронные*. На самом деле ответы контроллер может посылать только после соответствующего запроса, но их может быть несколько. Чтобы вписать соответствующую схему в полностью синхронный протокол Ајах, контроллеру необходимо в интерфейс ViewControllerListener посылать еще одну команду – *окончание ответа*. Тогда сервлет будет накапливать ответы и посылать их единым сообщением.

Использование фреймворка DWR [47-9] скрывает излишние подробности Ајах при программировании на Java:

- на сервере пишется и объявляется класс с определенным именем, методы которого напрямую вызываются на клиентской стороне из JavaScript;
- DWR компилирует интерфейсы этих методов в вызовы аналогичных методов в JavaScript с передачей параметров. Например, если на сервере есть класс XXX с методом AAA(int b, String c), то его можно вызвать напрямую из JavaScript в виде XXX.AAA(5, "fff");
- для обработки ответа в вызове может быть указано имя функции, которая асинхронно вызывается при получении ответа и содержит параметр – ответ, например строку.

На рис. 72-28 изображено решение, позволяющее web-клиенту максимально использовать код тонкого клиента с webAPI.

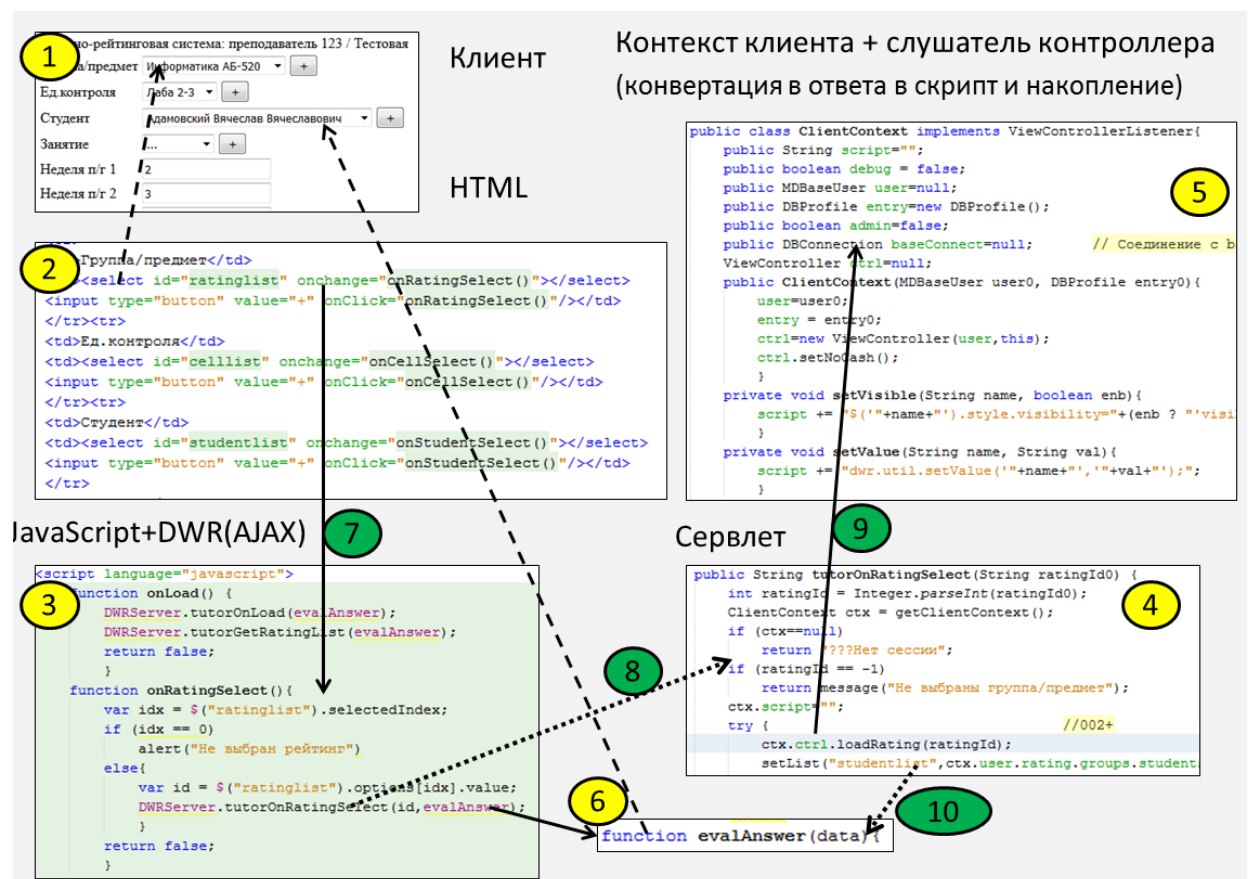


Рис. 72-28. Web-клиент с повторным использованием кода

Клиент строится и работает следующим образом:

- разметка и имена элементов управления копируются из java-файла с формой в html-код (1);
- по событию в форме (2) планируется вызов функции (7) в коде JavaScript (3). Функция средствами DWR вызывает метод в классе сервлета DWRServer (4) через одноименный метод, сгенерированный в JavaScript (8). Ему передаются параметры в формате строки, в т.ч. идентификаторы объектов;
- метод сервера получает сессию и объект-контекст (5), с которым связаны контроллер и слушатель его событий;
- код метода сервера (4) при обращении к контроллеру (9) *изоморфен* коду сервлета тонкого клиента с WebAPI, т.е. формально откорректирован с учетом изменения окружения, его исполняющего;
- объект-контекст является одновременно слушателем событий контроллера. Обработывая их, он пишет результаты в формате текстовой строки *в виде текста скрипта для клиента на JavaScript*;
- по окончании обращения к контроллеру накопленный текст скрипта возвращается клиенту (10);
- клиент, получая строку ответа, интерпретирует содержащийся в нем скрипт методом *eval* (6), который вызывается в функции асинхронного завершения запроса.

Параллелизм. Синхронизация

Специальных требований к параллелизму не предъявляется. В элементах графического интерфейса исполнение продолжительных операций происходит в фоновом режиме, при этом элементы управления не блокированы, но и не позволяют выполнять действия над данными, обрабатываемыми в фоне (*мягкая* блокировка).

Структура кода и объемные показатели

Для удобства повторного использования кода значительная часть классов выделена в *ядро (core)* - уровни контроллера для приложения преподавателя, бизнес-логика, классы DAO, коммуникации, коннекторы к БД, интерфейсы, что составляет 37% кода в строках исходного текста (SLOC) (рис. 72-29).

Компонента	SLOC	%	Пакет	Классов	Строк	Назначение
Core (ядро)	7816	37%	brs	2	325	Параметры настройки приложений
			brs.connect	4	828	Коннекторы к БД
			brs.model	29	3412	Бизнес-объекты
			brs.database	34	1207	Табличные объекты БД
			brs.ftplibclient	4	880	ftp-клиент хранилища - не используется
			brs.controller	3	646	Контроллер бизнес-логики "преподаватель"
			brs.xml	8	244	Команды-ответы протокола WebAPI
			brs.interfaces	5	143	Интерфейсы компонент
			brs.cui	5	131	Объекты протокола ЦИУ
Android	4044	19%	brs.view	26	3755	Уровень представления (экраны)
			brs.sqlite	2	289	Коннектор БД SQLite (Android) - не используется
Desktop (все приложения)	7980	38%	brs.javaview	40	7748	Уровни представления и бизнес-логики (экраны) - все приложения, "преподаватель" - только представление
			view	1	232	Таблица с прокруткой
WebAPI	684	3%	brs.web	1	684	Сервлет WebAPI для тонкого клиента
	541	3%	brs.web	1	541	Java - серверная часть
Web					400	Клиентская часть (HTML+JavaScript)
Всего	21065					

Рис.7-29. Объемные показатели проекта в SLOC